

22.07.26 Experiment-11

Aim:

To illustrate the concept of multithreading using a C program.

Code:

```
#include <stdio.h>
#include <pthread.h>
void * display (void * arg)
{
    printf("Thread is running\n");
    return NULL;
}

int main()
{
    pthread_t t1;
    pthread_create(&t1, NULL, display, NULL);
```

... pain ...
... school ...
... of ...

~~... is ...~~
~~... ..~~

~~...~~
... the concept of ...
... illustrating ... a C
program.

... ..
... ..

~~...~~
To simulate the Dining Philosophers
problem using a C program.

~~...~~
... ..

... ..

... ..
... ..

... ..

... ..
... ..

philof (Philosopher 1. d is Eating \n", i);

3
return 0;

3
Output:

Philosopher 1 is Thinking
" " Eating
" 2 4 Thinking
" " 4 Eating
" 3 4 Thinking
" " 4 Eating
" 4 4 Thinking
" 4 4 Eating
" 5 4 Thinking
" 11 11 Eating

~~Result:~~

Thus, the Dining Philosophers problem was successfully simulated using C program.

20/2/20

Experiment 12

Aim:

To implement First Fit, Best Fit, and Worst Fit memory allocation strategies using C.

Code:

#include <stdio.h>

int main() {

int block[S] = {100, 500, 200, 300, 600};

int process[P] = {212, 417, 112, 426};

printf("Memory allocation strategies\n");

printf("First Fit: P₁ → B₂, P₂ → B₅, P₃ → B₂,
P₄ → not allocated\n");

printf("Best Fit: P₁ → B₄, P₂ → B₂, P₃ → B₃,
P₄ → B₅ \n");

printf("Worst Fit: P₁ → B₅, P₂ → B₂, P₃ → B₅,
P₄ → Not Allocated\n");

return 0;

}

Notes:

Memory Allocation Strategies

~~First Fit~~ First Fit: P₁ → B₂, P₂ → B₅, P₃ → B₂, P₄ → not allocated.

Best Fit
Worst Fit
alloc

Result

First

Best

21/2

Alloc

Worst

Code

Belk Ext: $P_1 \rightarrow B_1, P_2 \rightarrow B_2, P_3 \rightarrow B_3, P_4 \rightarrow B_5$
Wolk Ext: $P_1 \rightarrow B_5, P_2 \rightarrow B_2, P_3 \rightarrow B_5, P_4 \rightarrow B_5$
allocated.

Result:
A C program successfully demonstrated the
First Fit, Best Fit & Wolk Ext memory allocation
strategies.

24-7-2020 Experiment - 14

Aim:

To construct a C program to organize file
using a single level directory.

Code:

```
#include <stdio.h>

int main () {
    int n, i;
    char file[10][20];
    printf("Enter number of files: ");
    scanf("%d", &n);
    printf("Enter file names: \n");
    for (i=0; i<n; i++) {
        scanf("%s", file[i]);
    }
    printf("\n Files in single level directory: \n");
```

File: 0; recursive &
path: C:\d, 7.81u", 1st, file [0]

26.8

Aim:

To
create

Code

It

is

Input:

Number of files: 3

Each file named:

file 1.txt

file 2.txt

image.jpg

Files in Single Level Directory:

1. file 1.txt

2. file 2.txt

3. image.jpg

Result:

The program to create files using a
Single Level Directory was created successfully, and
the list of files was displayed.

Aim:

To design a C program to organize files using a two level directory structure.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int ulevels, files, i, j;
```

```
    char ulev[10][20], file[10][10][20];
```

```
    printf("Enter number of ulevels: ");
```

```
    scanf("%d", &ulevels);
```

```
    for (i=0; i<ulevels; i++) {
```

```
        printf("Enter ulev name: ");
```

```
        scanf("%s", ulev[i]);
```

```
        printf("Enter number of files: ");
```

```
        scanf("%d", &files);
```

```
        printf("Enter file names: \n");
```

```
        for (j=0; j<files; j++) {
```

```
            scanf("%s", file[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\n Two Level Directory Structure: \n");
```

1. The first part of the paper is devoted to a discussion of the general principles of the theory of the structure of the atom.

1942

1. *Handwritten text, likely bleed-through from the reverse side of the page.*

10. 10.10.2019

Full Class

2000-2001

11-11-11

G. Ant
Uld: Bob
C. Ant

Result:

~~The~~ C program to organize files using a Two Level Directory Structure was executed successfully.

27-7-2026 Experiment-16

Aim:

To develop a C program to implement Random Access File for storing employee details.

Code:

```
#include <stdio.h>
```

```
struct Employee {
```

```
    int id;
```

```
    char name[20];
```

```
    float salary;
```

```
};
```

```
int main() {
```

```
    FILE *fp;
```

```
    struct Employee e;
```

```
    fp = fopen("employee.dat", "wb");
```

1st - 1st - 1st
2nd - 2nd - 2nd
3rd - 3rd - 3rd
4th - 4th - 4th
5th - 5th - 5th
6th - 6th - 6th
7th - 7th - 7th
8th - 8th - 8th
9th - 9th - 9th
10th - 10th - 10th

11th - 11th - 11th
12th - 12th - 12th
13th - 13th - 13th
14th - 14th - 14th
15th - 15th - 15th
16th - 16th - 16th
17th - 17th - 17th
18th - 18th - 18th
19th - 19th - 19th
20th - 20th - 20th

21st - 21st - 21st
22nd - 22nd - 22nd
23rd - 23rd - 23rd
24th - 24th - 24th
25th - 25th - 25th
26th - 26th - 26th
27th - 27th - 27th
28th - 28th - 28th
29th - 29th - 29th
30th - 30th - 30th

31st - 31st - 31st
32nd - 32nd - 32nd
33rd - 33rd - 33rd
34th - 34th - 34th
35th - 35th - 35th
36th - 36th - 36th
37th - 37th - 37th
38th - 38th - 38th
39th - 39th - 39th
40th - 40th - 40th

41st - 41st - 41st
42nd - 42nd - 42nd
43rd - 43rd - 43rd
44th - 44th - 44th
45th - 45th - 45th
46th - 46th - 46th
47th - 47th - 47th
48th - 48th - 48th
49th - 49th - 49th
50th - 50th - 50th

51st - 51st - 51st
52nd - 52nd - 52nd
53rd - 53rd - 53rd
54th - 54th - 54th
55th - 55th - 55th
56th - 56th - 56th
57th - 57th - 57th
58th - 58th - 58th
59th - 59th - 59th
60th - 60th - 60th

61st - 61st - 61st
62nd - 62nd - 62nd
63rd - 63rd - 63rd
64th - 64th - 64th
65th - 65th - 65th
66th - 66th - 66th
67th - 67th - 67th
68th - 68th - 68th
69th - 69th - 69th
70th - 70th - 70th

71st - 71st - 71st
72nd - 72nd - 72nd
73rd - 73rd - 73rd
74th - 74th - 74th
75th - 75th - 75th
76th - 76th - 76th
77th - 77th - 77th
78th - 78th - 78th
79th - 79th - 79th
80th - 80th - 80th

Result:
~~The C program for implementing a Random Access File to process employee details was executed successfully.~~

27.7.26 Experiment - 17

Aim:

To illustrate Deadlock Avoidance by Simulating Banker's Algorithm using C.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int available = 3;
```

```
    int max = 7;
```

```
    int allocated = 4;
```

```
    int need;
```

```
    need = max - allocated;
```

```
    printf("Available Resources : %d\n", available);
```

```
    printf("Maximum Need : %d\n", max);
```

```
    printf("Allocated Resources : %d\n", allocated);
```

```
    printf("Remaining Need : %d\n", need);
```

```
    if (need <= available)
```

```
        printf("Safe State - Resources can be allocated
```

else

```
        printf("Unsafe State - Deadlock may occur.
```

Module 2

Page 1

Instantaneous potential
maximum value
potential difference
potential difference

Self-inductance can be illustrated..

Self-inductance is the property of a coil which opposes the change in current flowing through it. It is a scalar (al) quantity.

Self-inductance is denoted by L.

It is defined as the ratio of the change in magnetic flux to the change in current which produces it.

It is denoted by L.

It is denoted by L.

It is denoted by L.

It is denoted by L.

mutex --;
empty --;
full ++;
mutex ++;

printf ("After Producer : Empty = %d Full = %d\n",
empty, full);

mutex --;
full --;
empty ++;
mutex ++;

printf ("After Consumer : Empty = %d Full = %d\n",
empty, full);
return 0;

}

Output:

Initial : Empty = 5 Full = 0

~~After Producer~~ : Empty = 4 Full = 1

~~After Consumer~~ : Empty = 5 Full = 0

Result:

~~The~~ C program successfully simulates the
Producer-Consumer problem using semaphores,
demonstrating the synchronization between producer
and consumer processes.

24.7.26

Experiment - 19

Aim:

To design a C program to implement pthread synchronization using mutex lock.

Code:

```
#include <stdio.h>
#include <pthread.h>
pthread_mutex_t lock;
int count = 0;
void *display (void *arg) {
    pthread_mutex_lock(&lock);
    count++;
    printf("Count = %d\n", count);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main () {
    pthread_t t1, t2;

    pthread_mutex_t int t (pthread_mutex_t);
    pthread_create (&t1, NULL, display, NULL);
    pthread_create (&t2, NULL, display, NULL);
```

```
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_mutex_delay(&mutex);
return 0;
```

3.

Output:

Count = 1
Count = 2

Result:

The C program successfully implements process synchronization using mutex lock, ensuring that only one thread access the shared resource at a time.

29.7.26 Experiment - 20

Aim:

To convert a program to simulate the reader-writer problem using semaphores.

Code:

```
#include <stdio.h>
int main() {
    int readcount = 0;
    int data = 100;
    readcount++;
```



```

printf("Reader reads data = %d\n", data);
readcount--;
data = 200;
printf("Writer updates data = %d\n", data);
return 0;
}

```

Output:

Reader reads data = 100
 Writer updates data = 200

Result:

The C program successfully simulates the Reader-Writer problem using semaphores demonstrating synchronized access to shared data.